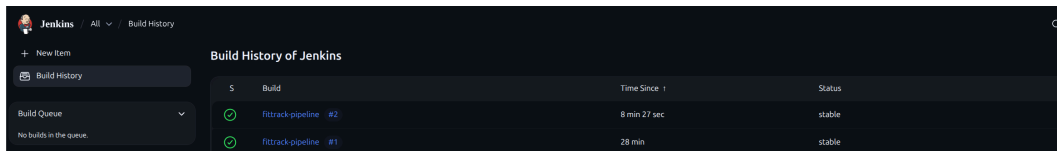


KUBERNETES CI/CD WITH HELM AND MONITORING

By Ahmed Tetteh | May 2026



Build History of Jenkins			
	Build	Time Since	Status
5	fittrack-pipeline #2	8 min 27 sec	stable
	fittrack-pipeline #1	28 min	stable

Project Vision: Building a Production-Grade DevOps Stack

What this project set out to achieve

In this project, I'm building a multi-page FitTrack app deployed in kubernetes cluster with Helm deployed monitoring stack and a Jenkins CI/CD pipeline for rapid deployment of the app on every code change.

Setting Up Helm and the Monitoring Foundation

Goals for this step

In this step, I'm installing Helm and verifying its connectivity to my Kubernetes cluster running within Rancher Desktop, so that I can use it to create a monitoring stack of Prometheus and Grafana.

```
king@king-TFG5577XG:~$ helm search repo kube-prometheus-stack
NAME                                CHART VERSION  APP VERSION  DESCRIPTION
prometheus-community/kube-prometheus-stack  84.3.0         v0.90.1      kube-prometheus-stack collects Kubernetes manif...
```

Understanding Helm's core concepts

A chart is a package of Kubernetes YAML templates and default configuration. Think of it like a blueprint for deploying an application.

A repo (repository) is where charts are published and downloaded from. It works like npm for JavaScript or pip for Python.

A release is a specific installation of a chart on your cluster. You can install the same chart multiple times, and each installation is its own release with its own name.

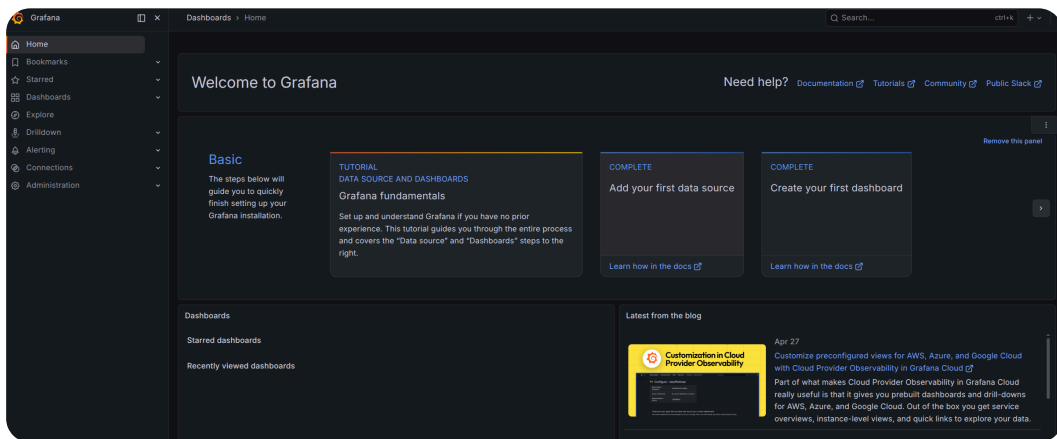
Deploying Prometheus and Grafana with Helm

Goals for this step

In this step, I'm creating custom Helm values for my Kube-prometheus monitoring stack for local development before deploying it within my cluster.

Key configuration: ServiceMonitor discovery

This setting ensures that Prometheus can discover all other resources in other namespaces, not just the default ones.



Grafana login credentials

I logged in with the custom adminPassword which I configured in the monitoring-values.yaml file when starting up the chart within the cluster.

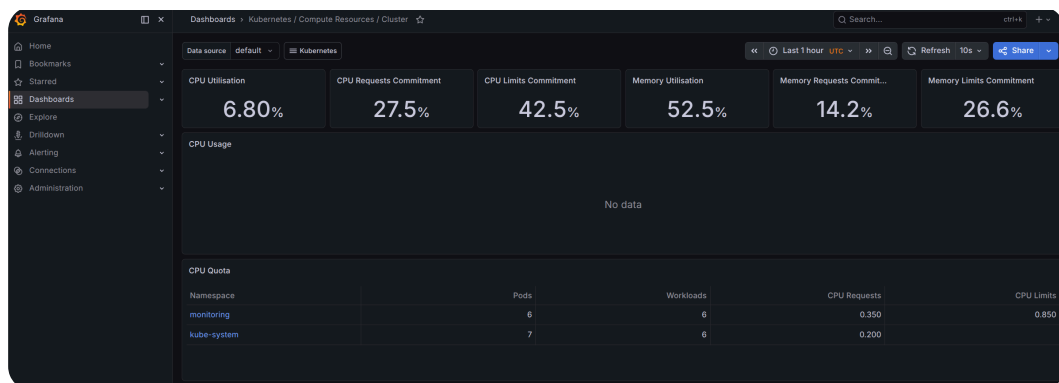
Understanding the Helm Chart Architecture

Goals for this step

In this step, I'm exploring the resources created with my cluster on both the Helm and Kubernetes side so that I understand how all the resources link together.

Charts, values, and rendered manifests explained

The chart is a package of Go templates with placeholder variables. Your values fill in those placeholders with your specific configuration. The manifest is the final rendered YAML that Helm sends to Kubernetes.



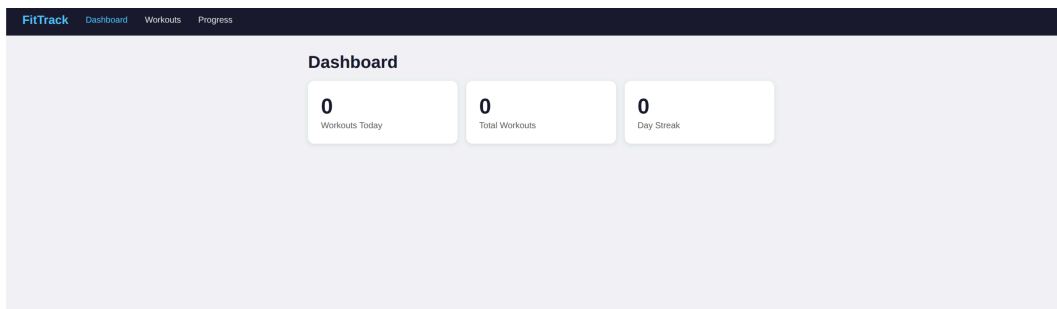
Prometheus Operator CRDs discovered

I found CRDs(Custom Resource Definition.) called ServiceMonitors and PrometheusRules. They are used for scraping metrics from various Kubernetes services and for defining various alerting conditions.

Building the FitTrack App with Prometheus Instrumentation

Goals for this step

In this step, I'm building the FitTrack app with Flask so that Prometheus can scrape some real application data from it.



How the /metrics endpoint powers Prometheus

The /metrics endpoint exposes the Flask app routes and request counts for tracking, which Prometheus uses to get real metrics on the application.

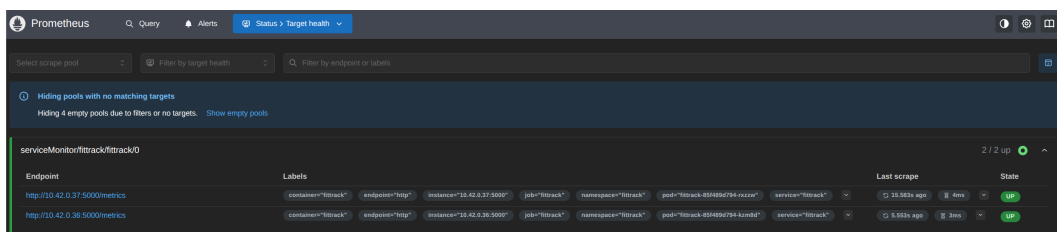
Deploying FitTrack to Kubernetes with Production Patterns

Goals for this step

In this step, I'm deploying the FitTrack app to the Kubernetes cluster and creating a Kubernetes Deployment manifest with resource limits and health checks.

Liveness vs. readiness probes

A liveness probe checks if your container is still running and healthy. If it fails, Kubernetes restarts the container automatically, while a readiness probe checks if your container is ready to accept traffic. If it fails, Kubernetes removes the pod from the Service's endpoints (stops sending traffic to it), but does not restart it.



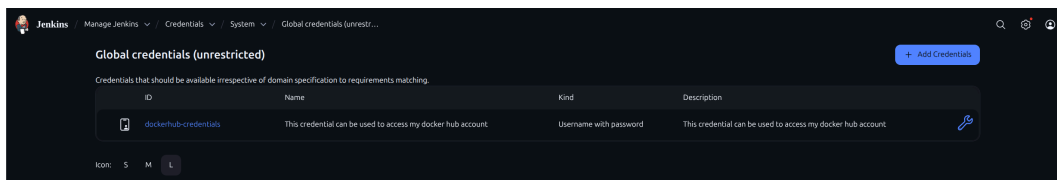
How ServiceMonitor enables automatic metrics discovery

The ServiceMonitor tells Prometheus to scrape for metrics from some particular services by specifying an endpoint(/metrics) to collect the metrics.

Automating Deployments with a Jenkins CI/CD Pipeline

Goals for this step

In this step, I'm setting up a Jenkins pipeline to automate my build process for the app.



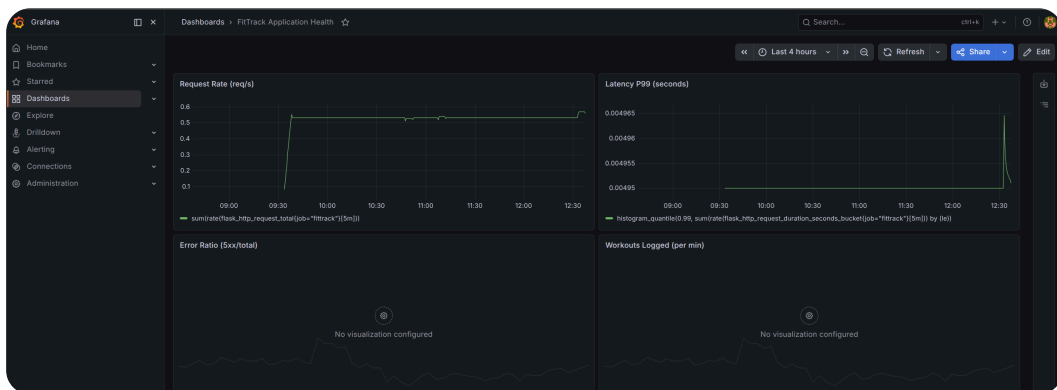
Breaking down each pipeline stage

The Checkout stage pulls the latest code from whatever source control Jenkins is pointed at. The Build stage uses the Docker Pipeline plugin to run `docker.build()`, which builds my image from the Dockerfile. The Push stage authenticates with Docker Hub using my stored credentials, then pushes the image with both the build number tag and the latest tag. The Deploy stage runs `"kubectl set image"` to update the running Kubernetes Deployment to the new image tag. It then waits for the rollout to finish successfully.

Building Custom Dashboards and Alerting Rules

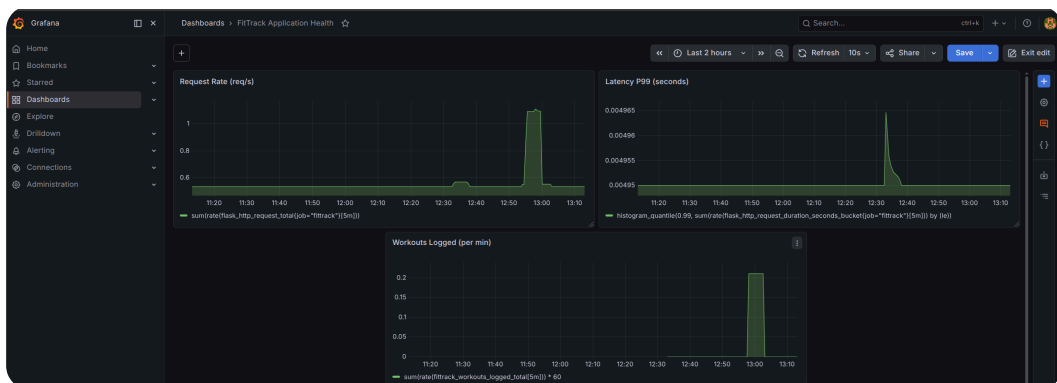
Goals for this step

In this step, I'm building a PrometheusRule that can fire an alert when a threshold is reached, so that I can monitor the app's performance through a custom Grafana dashboard.



PrometheusRule alert logic explained

The rule monitors when more than 5% of requests return 5xx errors for 2 continuous minutes, and waits 2 minutes before firing to prevent noisy alerts from brief, one-off failures.



Secret Mission: Packaging FitTrack as a Reusable Helm Chart

```
king@king-TFG5577XG:~$ k get pods
NAME                                READY   STATUS    RESTARTS   AGE
fittrack-85f489d794-qcdxg          1/1     Running   0           5m2s
fittrack-85f489d794-qfng9          1/1     Running   0           5m2s
fittrack-85f489d794-x4lwd          1/1     Running   0           5m2s
king@king-TFG5577XG:~$ k get deployments.apps
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
fittrack  3/3     3             3           5m11s
```

How Helm templating and value overrides work

In this project extension, I learned that `{{ .Values }}` is used as a placeholder that the templates utilize to configure settings and `--set` allows me to modify the values within my `values.yaml` files before installation

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Kubernetes, Helm, Jenkins, Prometheus, Grafana and Python. Key concepts I learnt include how to:

- Deploy a full Prometheus and Grafana monitoring stack using Helm, complete with custom values, ServiceMonitors for automatic metrics discovery, and PrometheusRule alerts.
- Build a Jenkins CI/CD pipeline that automates building, pushing, and deploying a containerized Flask application to Kubernetes on every code change.
- Package FitTrack as a reusable Helm chart with configurable values for replicas, image tags, and resource limits.

Time and challenges

This project took me approximately 8 hrs (including setup and background studies). The most challenging part was when I tried to automate everything using the CI/CD pipelines, but kept encountering errors - due to Jenkins not being able to discover the kubectl binary and execute the commands within the cluster.

Looking ahead

I did this project today to learn how to build production-grade DevOps pipelines from scratch.